De Revisión Integral (Python y C++)

Calculadora de edad exacta con validación del calendario real

Alexis Maier

E.E.S.T. N° 1 "Eduardo Ader" — Vicente López
Laboratorio de Programación (LPR) — 5° Año 3° División — Grupos A-B

Profesores: Mansilla Muñoz York Elías / Ganduglia Yamil
Fecha: 01 de octubre de 2026

Introducción

La primera actividad del ciclo lectivo consistió en una revisión integral de los dos lenguajes retomados en la carrera: Python y C++. La consigna planteaba el desarrollo de una calculadora de edad que no se limitara a restar dos años, sino que validara la existencia real de la fecha ingresada dentro del calendario, incluyendo el tratamiento de los años bisiestos y la longitud variable de los meses. Se implementaron dos versiones independientes del mismo algoritmo, una en cada lenguaje, con el propósito de comparar las diferencias de sintaxis, de manejo de la entrada por consola y de control de errores entre ambos paradigmas.

Desarrollo

Algoritmo de validación del calendario

El núcleo de la solución se apoya en dos funciones auxiliares. La primera, **esBisiesto(anio)**, determina si un año es bisiesto mediante la regla gregoria clásica: es bisiesto si es divisible por 4, siempre que no sea divisible por 100, salvo que además sea divisible por 400. Esta condición descarta los años seculares, como 1900, y mantiene los milenarios, como 2000. La segunda función, **esFechaValida(d, m, a)**, encadena los filtros de consistencia: rango de años comprendido entre 1900 y 2026, mes entre 1 y 12, día entre 1 y 31, exclusión del día 31 en los meses de 30 días (abril, junio, septiembre y noviembre) y tratamiento especial de febrero, que admite 29 días únicamente cuando el año resultante es bisiesto.

Cálculo de la edad exacta

Una vez superada la validación, el cálculo de la edad no se resuelve con una simple resta, porque eso produciría un resultado incorrecto durante todo el mes previo al cumpleaños. La regla implementada es la siguiente: primero se calcula la diferencia entre el año actual y el año de nacimiento; luego, si el mes de nacimiento es posterior al mes actual, o si se está en el mismo mes pero todavía no llegó el día del cumpleaños, se descuenta una unidad. La fecha de referencia utilizada en el ejercicio fue el 08/07/2026.

Diferencias entre las dos implementaciones

La versión en C++, archivada como *proyecto_cpp/Act 1 - calculadoraEdad.cpp*, lee la fecha en una única instrucción mediante tres extractores separados por espacios (*cin >> diaN >> mesN >> anioN*) y señala el error de datos con un *return 1*, que el sistema operativo interpreta como código de salida distinto del éxito. La versión en Python, archivada como *proyecto_python/Act 1 - calculadoraEdad.py*, solicita cada componente por separado con *input()* y envuelve toda la lógica en un bloque *try / except ValueError*, que captura la conversión de texto a entero inválida. Ambos programas imprimen la misma cabecera con los integrantes y los mismos mensajes de resultado, lo que permite verificar que la lógica es equivalente.

Regla de no plagio

Siguiendo la regla obligatoria de la consigna, la salida de consola de ambos programas fue personalizada con los nombres y apellidos reales del grupo antes de la compilación y de la entrega.

Cuestionario Técnico de Control

1. Validación de la fecha

Validar la fecha antes de operar evita obtener resultados silenciosamente incorrectos. Si se aceptara, por ejemplo, el 31 de abril, el cálculo de la edad no fallaría de forma visible, pero partiría de un dato inexistente, lo que degrada la credibilidad del programa. La validación actúa como una barrera de integridad entre la entrada del usuario y la lógica de negocio.

2. Regla del año bisiesto

Un año es bisiesto si es múltiplo de 4 y no es múltiplo de 100, o bien si es múltiplo de 400. La primera condición cubre la regla general; la excepción sobre los múltiplos de 100 corrige el desvío del calendario juliano, y la salvedad sobre los múltiplos de 400 mantiene la corrección del calendario gregoriano. En consecuencia, 1900 no fue bisiesto y 2000 sí lo fue.

3. Código de salida en C++

En C++, *return 1* dentro de *main* finaliza el programa y devuelve un entero al sistema operativo. El valor 0 indica ejecución exitosa y cualquier otro valor señala error, lo que permite que un script externo detecte el fallo. En Python la misma idea se resuelve con *sys.exit(1)* o simplemente con el bloque *else*, ya que el intérprete no expone el código de salida por omisión.

4. Criterio para descontar un año

Se descuenta una unidad cuando el cumpleaños todavía no ocurrió en el año en curso. Esto equivale a comparar el par (mes, día) de nacimiento contra el par (mes, día) actual: si el par de nacimiento es posterior, la edad se reduce en uno. De este modo el resultado coincide exactamente con la edad que figura en el documento de identidad.

5. Autoevaluación

El mayor desafío fue replicar en Python la lógica del programa en C++ sin recurrir a conversiones implícitas de tipo, lo que exigiría comprender bien la diferencia entre el lenguaje compilado y el interpretado. Como mejora pendiente quedó reemplazar la fecha de referencia fija por la obtención de la fecha del sistema operativo.

Conclusión

La actividad confirmó que el dominio de un algoritmo no está terminado hasta que el programa rechaza las entradas que violan las reglas del problema. Incorporar la validación del calendario y la regla del cumpleaños transformó un ejercicio aritmético en una solución robusta. El paralelismo entre C++ y Python permitió contrastar dos modelos de ejecución sobre el mismo algoritmo: la verificación estricta de tipos y el control explícito del flujo en el lenguaje compilado, frente al tipado dinámico y el manejo de excepciones en el lenguaje interpretado. Ambos productos constituyen la base conceptual sobre la que se apoyan las actividades de punteros, estructuras y modularidad que siguen en la secuencia.

Referencias

Python Software Foundation. (s. f.). *Built-in functions*. Documentación de Python 3. https://docs.python.org/3/library/functions.html

Python Software Foundation. (s. f.). *Errors and exceptions*. Documentación de Python 3. https://docs.python.org/3/tutorial/errors.html

cppreference.com. (s. f.). *Program termination*. Referencia de C++. https://en.cppreference.com/w/cpp/language/return

Microsoft Learn. (s. f.). *Operadores de entrada y salida (C++)*. https://learn.microsoft.com/es-es/cpp/standard-library/iostream